

AV2 - CB13 - Programação 1 - 2026.1 - Prof. Marcelo Maia

Questão 1 [1,5 pontos]: Analisador de Logs de Sistema

Um sistema de segurança registra as tentativas de acesso em um arquivo de texto. Você deve criar um programa que processe esses registros para identificar acessos suspeitos. Um acesso é considerado suspeito se a senha tentada tiver menos de 6 caracteres ou se o nome de usuário contiver a palavra "admin" (em qualquer variação de maiúsculas/minúsculas).

Formato da Entrada:

1. O nome de um arquivo que contém o log de acessos.
2. O arquivo será composto por múltiplas linhas de texto, cada uma contendo um registro no formato: `Usuario:Senha`.

Formato da Saída:

- Para cada acesso suspeito, imprimir imediatamente: `ALERTA: Tentativa suspeita detectada para usuário '{usuario}'`.
- Ao final, imprimir um resumo formatado: `Total de tentativas: {total_tentativas} | Suspeitas: {total_suspeitos}`.

Exemplos:

Entrada	Conteúdo do Arquivo	Saída
<code>logs_seguranca.txt</code>	<code>ana:senha123 adminDB:1234567 carlos:12345 joao:seguro1</code>	<code>ALERTA: Tentativa suspeita detectada para usuário 'adminDB'. ALERTA: Tentativa suspeita detectada para usuário 'carlos'. Total de tentativas: 4 Suspeitas: 2.</code>
<code>acessos.txt</code>	<code>maria:123 Admin:12</code>	<code>ALERTA: Tentativa suspeita detectada para usuário 'maria'. ALERTA: Tentativa suspeita detectada para usuário 'Admin'. Total de tentativas: 2 Suspeitas: 2.</code>

Questão 2 [1,5 pontos]: Análise de Dados de Redes Sociais

Uma rede social deseja criar um sistema simples de recomendação de interesses. O programa deve ler os hobbies de vários usuários e armazená-los. Em seguida, o programa deve comparar os hobbies de dois usuários específicos e exibir o que eles têm em comum e o que um tem que o outro não tem.

Formato da Entrada:

1. Um inteiro **N** indicando a quantidade de usuários a serem registrados.
2. Nas **N** linhas seguintes, os dados no formato: **Nome,hobbie1,hobbie2,hobbie3, ...,hobbieN**
3. Duas linhas finais contendo os nomes de dois usuários (**UsuarioA** e **UsuarioB**) que serão comparados.

Formato da Saída:

- Uma linha com os interesses em comum: **Interesses em comum: {hobbieX, hobbieY, ...}**.
- Uma linha com os interesses exclusivos do UsuarioA: **Exclusivos de {UsuarioA}: {hobbieZ, ...}**.
- Uma linha com os interesses exclusivos do UsuarioB: **Exclusivos de {UsuarioB}: {hobbieK, ...}**.
- *Nota: Se um ou ambos usuários não forem encontrados, imprimir: **Erro: Usuário não cadastrado**.*

Exemplos:

Entrada	Saída
<pre> 3 Ana, leitura, cinema, corrida Beto, futebol, cinema, natação Carlos, pintura Ana Beto </pre>	<pre> Interesses em comum: {'cinema'}. Exclusivos de Ana: {'corrida', 'leitura'}. Exclusivos de Beto: {'natação', 'futebol'}. </pre>

2 Joao,games Maria,filmes Joao Lucas	Erro: Usuário não cadastrado.
--	-------------------------------

Questão 3 [2,0 pontos]: Processamento Seguro de Arquivos

Você deve criar uma função chamada `processar_transacoes(nome_arquivo)` que tentará ler um arquivo de texto contendo dados financeiros de uma loja. Cada linha do arquivo deveria ter o formato `ID_Produto,Valor`. O programa deve somar todos os valores e gravar o total em um novo arquivo chamado `resultado_final.txt`. Como os arquivos podem estar corrompidos, sua função deve tratar proativamente a ausência do arquivo e a conversão de textos não numéricos para `float`.

Formato da Entrada:

1. O nome do arquivo a ser processado (ex: "vendas.txt").

Formato da Saída:

- Se o arquivo não existir, imprimir: `Erro: Arquivo '{nome_arquivo}' não encontrado.`
- Se algum valor não puder ser convertido para float, imprimir: `Aviso: Linha ignorada devido a dados corrompidos.`
- Se a leitura for bem-sucedida, criar o arquivo `resultado_final.txt` contendo o texto: `Total arrecadado: R$ {soma_total}.`

Exemplos:

Entrada	Arquivo de Entrada	Saída	Arquivo de Saída (resultado_final.txt)
<code>inexistente.txt</code>	<i>(arquivo não existe)</i>	<code>Erro: Arquivo 'inexistente.txt' não encontrado.</code>	<i>(arquivo não criado)</i>

vendas.txt	P01,50.00 P02,abc P03,150.50	Aviso: Linha ignorada devido a dados corrompidos.	Total arrecadado: R\$ 200.50
------------	------------------------------------	---	------------------------------

Questão 4 [2,0 pontos]: Busca Rápida

Um sistema de logística possui uma lista contendo milhões de códigos de rastreamento de pacotes, já ordenados de forma crescente. Você deve implementar o algoritmo de busca binária estritamente na forma de uma função recursiva.

Formato da Entrada:

1. Uma linha com números inteiros ordenados separados por espaço (a lista completa).
2. Uma linha com o número inteiro X (o alvo a ser buscado).

Formato da Saída:

- Se encontrado, imprimir: Pacote {X} localizado no índice {índice}.
- Se não encontrado, imprimir: Pacote {X} não consta no sistema.

Exemplos:

Entrada	Saída
10 25 30 45 50 80 45	Pacote 45 localizado no índice 3.
5 15 35 60 20	Pacote 20 não consta no sistema.

Questão 5 [3,0 pontos]: Simulador de Robôs Exploradores

Uma agência espacial está simulando o envio de robôs (também conhecidos como “Zords”) a Marte. Defina uma classe **Robo** cujas instâncias representam Zords.

1. **Atributos:** **nome**, **coord_x**, **coord_y** e **bateria** (iniciando sempre em 100.0).
2. **Método mover(dx, dy):** Atualiza as coordenadas **x** e **y** somando **dx** e **dy**. Cada movimento consome carga da bateria. Assuma que o consumo é de 1% da capacidade da bateria por unidade de distância percorrida e que o movimento sempre é realizado em uma linha reta sobre uma superfície plana. Se a carga da bateria for insuficiente para realizar o movimento solicitado, o método deve levantar uma exceção do tipo **Exception** com a mensagem **Bateria insuficiente!** e não realizar o movimento.
3. **Método __str__:** Deve retornar o estado do robô no formato: **[{nome}] Pos: ({x}, {y}) | Bateria: {bateria}%**
Nota: as coordenadas **x** e **y** devem ser representadas com precisão de 1 casa decimal e a carga da bateria arredondada para um valor inteiro.
4. **Operação de acoplamento de robôs:** Os Zords podem se unir em uma formação conhecida como “Megazord”. O operador **+** entre dois robôs deve criar e retornar um novo robô no ponto médio do segmento de reta entre suas posições. O nome do Megazord formado é uma junção dos nomes dos Zords unidos (ex: **"RoboA" + "RoboB" → "RoboA-RoboB"**) e a carga de sua bateria será a soma das cargas dos robôs (limitada a 100).

Formato da Entrada: O programa principal deve instanciar diversos robôs recebendo os dados do terminal.

1. Uma linha com 2 valores **N** e **M** separados por um espaço, respectivamente o nº de robôs e o nº de operações a serem realizadas.
2. Cada uma das **N** linhas seguintes conterá os dados dos robôs a serem instanciados, no formato **Nome, X, Y**.
3. Cada uma das **M** linhas seguintes conterá os dados de uma operação a ser executada, podendo ser:
 - **MV Nome, dx, dy** que representa um movimento a ser aplicado ao robô com o nome indicado.
 - **MZ Nome1, Nome2** que representa a formação de um Megazord unindo os robôs com os nomes indicados.

Formato da Saída:

- Para cada robô instanciado, imprimir: **Robô em operação: {estado do robô}**
- Para cada operação solicitada na entrada, imprimir o resultado:
 - Se for uma movimentação, imprimir: **Robô movido: {estado do robô após movimento}**
ou (em caso de falha): **Robô {nome} não pôde ser movido: {causa da falha}**
As possíveis causas de falha são: não encontrar um robô com o nome especificado; a carga da bateria do robô ser insuficiente.

- Se for uma formação de Megazord, imprimir: **Megazord formado: {estado do Megazord formado}**
Observação: sempre que um Megazord é formado, os Zords individuais que se uniram para formá-lo deixam de existir separadamente (todas as referências a esses objetos devem ser descartadas).

Exemplos:

Entrada	Saída
2 4 R2D2,0,0 C3P0,10,0 MV R2D2,3,4 MZ R2D2,C3P0 MV C3P0,-50,99 MV R2D2-C3P0,-6.5,-2	Robô em operação: [R2D2] Pos: (0.0, 0.0) Bateria: 100% Robô em operação: [C3P0] Pos: (10.0, 0.0) Bateria: 100% Robô movido: [R2D2] Pos: (3.0, 4.0) Bateria: 95% Megazord formado: [R2D2-C3P0] Pos: (6.5, 2.0) Bateria: 100% Robô C3P0 não pôde ser movido: Robô não encontrado! Robô movido: [R2D2-C3P0] Pos: (0.0, 0.0) Bateria: 93%
1 1 Rover,0,0 MV Rover,101,0	Robô em operação: [Rover] Pos: (0.0, 0.0) Bateria: 100% Robô Rover não pôde ser movido: Bateria insuficiente!

Questão BÔNUS [0,5 ponto para quem não pôde participar da MFP nem da OBI]: Fila de Tarefas Dinâmica

Em vez de usar as listas nativas (`list`) do Python, que dependem de blocos contíguos de memória, você implementará do zero uma classe `FilaDinamica` baseada no conceito de Lista Encadeada. A classe deve ter os seguintes métodos:

1. `insere(descricao)`: Insere a string descricao no fim da fila.
2. `remove()`: Remove a primeira tarefa da fila e retorna sua descrição.

O nº de passos executados no pior caso por ambos os métodos descritos acima deve ser constante. A classe pode conter outras definições (classes, métodos, atributos) que você julgar necessárias.

Formato da Entrada:

1. O usuário digitará várias tarefas, uma por linha. A digitação encerra quando for inserida a string vazia (apenas apertar Enter). *Nota: A cada entrada, o programa deve chamar o método para inserir na FilaDinamica.*

Formato da Saída:

- Após a leitura e inserção de todas as tarefas na fila, o programa deve remover todas as tarefas e imprimi-las, uma por linha.

Exemplos:

Entrada	Saída
Estudar para a prova Limpar o quarto Fazer compras	Estudar para a prova Limpar o quarto Fazer compras
Revisar Python	Revisar Python